

Assignment 1:C programming

Course Code: ICS_2100

Group Name: BitMasters

Instructor's Name : John Wanaina

Group Members

1. Davidpraise Njinu Kimwaki - SCT211-0236/2024
2. Amoiti IvyneLisa Atieng - SCT211-0348/2024
3. Njanja Lawrence Kariuki - SCT211-0475/2024
4. Bethwel Kipruto Kangogo - SCT211-0387/2024
5. George Maina Wanjohi - SCT211-0345/2024
6. Winnie Mugoiri - SCT211-0238/2024
7. Andrew Chweya Okero Mokandu - SCT211-0251/2024
8. Willy Kaveva - SCT211-0240/2024
9. Caleb Mutai Yegon - SCT211-0025/2024
10. Kiarie Kariuki - SCT211-0002/2023

Assignment 1 Overview

This report details our implementation of four C programming problems focusing on:

1. Lucky Number Game
2. Student Grade Calculator
3. Credit Limit Checker
4. Employee Tax Calculator

Problem Implementations

1. Lucky Number Game

Problem Description

A program that implements a game where users:

- Input number of rounds
- Enter lucky numbers for each round
- Score points based on division remainder with secret number
- Win/lose based on final score

Implementation Approach

Our team approached this problem using the following methodology:

1. Ivy Declared this local variables;
 - rounds: To track remaining rounds
 - lucky_number: For user input
 - secret_number: Randomly generated for each round
 - remainder: For score calculation

total_score: To maintain running score
2. Caleb added Random Number Generation Implementation
 - Used srand() with time(0) seed for true randomization
 - Implemented rand()%10 + 1 to generate secret numbers between 1-10
 - Ensured different secret numbers for each round
2. Praise added Game Logic Implementation
 - While loop for controlling game rounds
 - Three scoring conditions:
 - Remainder = 0: Award 1 point (Draw)
 - Even remainder: Award 3 points (Win)
 - Odd remainder: Deduct 3 points (Loss)
 - Clear feedback messages for each outcome
3. User Interface Design
 - Input prompt for number of rounds
 - Round-by-round lucky number input
 - Informative messages for each scoring outcome
 - Final result display based on total score
4. Score Management
 - Running total maintained throughout game
 - Final evaluation for win/loss based on positive score
 - Appropriate victory or defeat message display

Our Approach

The game cleverly combines random number generation with basic number theory (divisibility and remainder properties) to create an engaging number guessing game.

2. Student Grade Calculator

Problem Description

The Student Grade Calculator is a program designed to streamline the process of managing and evaluating student academic performance. The program implements the following key features:

1. Data Input:
 - Captures student admission numbers and surnames for identification
 - Accepts three different marks per student
 - Uses a user-friendly command-line interface
2. Calculations:
 - Automatically calculates the average mark from the three input scores
 - Implements a standardized grading system based on average marks
3. Grading System:
 - A: Average mark ≥ 70
 - B: Average mark ≥ 60
 - C: Average mark ≥ 50
 - D: Average mark ≥ 40
 - F: Average mark < 40
4. User Experience:
 - Provides continuous processing capability for multiple students
 - Allows users to choose when to terminate the program
 - Displays comprehensive results including all input data and calculated grades

Our Implementation Approach

The program has been tested using the following methodology:

1. Input Validation Testing:
 - Tested admission number input with various formats
 - Verified surname input handling
 - Validated mark inputs within reasonable ranges (0-100)
2. Calculation Testing:
 - Verified average calculation accuracy with different combinations of marks
 - Tested boundary conditions for grade assignments (e.g., marks at 69.5, 70, 70.5)
3. Grade Assignment Testing:
 - Confirmed correct grade assignment for each grade boundary
 - Verified grade logic for extreme cases (very high and very low marks)
4. Loop Control Testing:
 - Tested continuation prompt with both 'Y' and 'y' inputs
 - Verified program termination with 'N' input
 - Checked for proper handling of invalid continuation inputs

3. Credit Limit Checker

Problem Description

The Credit Limit Checker is a program designed to help businesses manage customer credit purchases efficiently. The program implements a system to:

1. Process Multiple Customers:
 - Accepts a predefined number of total customers to process
 - Tracks each customer transaction individually
 - Maintains a running count of processed customers
2. Credit Management:
 - Validates purchases against preset credit limits
 - Ensures customers cannot exceed their credit limits
 - Provides immediate feedback on purchase eligibility
3. Purchase Calculation:
 - Calculates total purchase value based on quantity and price
 - Handles floating-point calculations for precise pricing
 - Allows quantity adjustments if credit limit is exceeded
4. User Interaction:
 - Provides clear prompts for all required inputs
 - Offers feedback messages for both successful and failed transactions
 - Allows quantity revision when credit limit is exceeded

Testing Strategy

1. Input Validation Testing:
 - Tested various total customer counts
 - Verified credit limit input handling
 - Validated price and quantity inputs for different scenarios
2. Calculation Testing:
 - Verified purchase value calculations (price × quantity)
 - Tested decimal price handling
 - Confirmed accurate comparison between credit limit and purchase value
3. Loop Control Testing:
 - Verified customer count tracking
 - Tested do-while loop functionality for quantity adjustments
 - Confirmed proper program termination after processing all customers

4. Employee Tax Calculator

Problem Description

The Employee Tax Calculator is a program designed to calculate tax rates based on employees' gross pay and number of dependants. The program implements the following features:

1. Input Collection:
 - Captures employee's gross pay amount
 - Records the number of dependants
 - Uses simple command-line interface for data entry
2. Tax Rate Determination: The program applies different tax rates based on two main factors:
 - Gross Pay Thresholds (10,000 and 20,000)
 - Number of Dependants (≥ 3 or < 3)
3. Tax Rate Structure: The program implements the following tax rate matrix:
 - 35%: Gross pay $> 20,000$ and dependants < 3
 - 25%: Gross pay $> 20,000$ and dependants ≥ 3
 - 15%: Gross pay $> 10,000$ and dependants < 3
 - 10%: Gross pay $> 10,000$ and dependants ≥ 3

Testing Strategy

1. Input Validation Testing:
 - Tested various gross pay amounts around threshold values
 - Verified dependant number input handling
 - Tested decimal handling for gross pay input
2. Boundary Testing:
 - Tested gross pay values at boundaries (10,000 and 20,000)
 - Verified tax rate assignment at dependant threshold (3)
 - Checked behavior at extreme values

Technical Achievements:

- Mastered fundamental C programming concepts including loops, conditional statements, and arithmetic operations through practical implementation of four distinct programs
- Developed proficiency in input validation techniques

- Gained hands-on experience with random number generation and mathematical algorithms in C
- Established effective version control practices using GitHub, learning essential commands and collaborative workflow procedures
- Developed strong teamwork capabilities by dividing tasks among 10 team members while maintaining code consistency
- Implemented code review practices that improved overall program quality and reliability
- Built communication skills through regular team meetings and documentation efforts
- Enhanced problem-solving abilities through tackling diverse programming challenges
- Developed debugging skills by identifying errors in our codes
- Gained practical experience in writing maintainable and well-documented code

Appendices

a. Link to our Github Repository
C-Programming Assignment1

C. Git Commit History
Git Commits History